

실시간 운영체제 기반 전자제어기 프로그래밍

Contents

실시간 운영체제 기반 전자제어기
프로그래밍

FreeRTOS 기본 개념

FreeRTOS 기본 개념

실시간 운영체제

□ Multitasking in Small Embedded Systems

○ FreeRTOS 커널 소개

- FreeRTOS는 실시간 커널(Real-Time Kernel)과 다양한 기능을 제공하는 모듈형 C 라이브러리로 구성된 소프트웨어이다.
 - FreeRTOS는 Richard Barry가 약 2003년에 처음 개발하였으며, 이후 Real-Time Engineers Ltd.가 주요 반도체 기업들과 협력하여 개발을 하였다.
 - 2016년부터 Amazon Web Services(AWS)가 FreeRTOS 프로젝트를 관리하고 있으며, FreeRTOS는 MIT 라이선스 기반의 오픈소스 소프트웨어로, AWS 사용 여부와 관계없이 누구나 사용할 수 있다.
 - FreeRTOS 커널은 특히 마이크로컨트롤러(MCU) 또는 소형 마이크로프로세서에서 동작하는 임베디드 실시간 시스템에 적합하며, 이러한 시스템에서는 일반적으로 Hard Real-Time과 Soft Real-Time 요구사항이 함께 존재한다.
- 연성 실시간 시스템과 경성 실시간 시스템은 마감시간의 준수 여부에 따라 달라진다.
 - Soft Real-Time 시스템 – 정해진 시간 제한(deadline)이 존재하지만, 이를 초과하더라도 시스템 전체가 완전히 실패하는 것은 아니다. 예를 들어 키보드 입력에 대한 반응이 다소 늦으면 사용자가 시스템을 답답하게 느낄 수 있지만, 시스템 자체를 사용할 수 없는 것은 아니다.
 - Hard Real-Time 시스템 – 정해진 시간 제한을 반드시 만족해야 하며, deadline을 초과하면 시스템의 심각한 실패로 이어질 수 있다. 예를 들어 자동차의 에어백(Airbag)이 충돌 센서의 입력에 너무 늦게 반응한다면 오히려 탑승자에게 더 큰 피해를 줄 수 있다.

실시간 운영체제

□ Multitasking in Small Embedded Systems (계속)

○ FreeRTOS 역할

- FreeRTOS 커널은 애플리케이션이 Hard Real-Time 요구사항을 만족할 수 있도록 하는 실시간 스케줄러(Real-Time Scheduler)이다.
- 애플리케이션을 여러 개의 독립적인 실행 흐름(Thread of Execution)으로 구성할 수 있도록 지원하며, FreeRTOS에서는 이러한 실행 단위를 일반적으로 Task라고 부른다.
- 단일 코어 프로세서에서는 한 시점에 하나의 Task만 실행할 수 있기 때문에, FreeRTOS 커널의 Scheduler는 각 Task에 부여된 Priority(우선순위)를 기반으로 어떤 Task를 실행할 것인지 결정한다.
 - 높은 우선순위 -> Hard Real-Time Task
 - 낮은 우선순위 -> Soft Real-Time Task

실시간 운영체제

□ Why Use an RTOS?

간단한 임베디드 시스템은 RTOS 없이도 충분히 구현할 수 있지만 시스템이 복잡해질수록 RTOS Kernel을 이용하는 것이 애플리케이션 구조와 실시간 요구사항 관리 측면에서 더 유리할 수 있다.

○ 시간 제어의 추상화 (Abstracting Away Timing Information)

➤ RTOS가 Task의 실행 시점과 시간 관리를 담당한다. 애플리케이션은 RTOS가 제공하는 시간 관련 API를 이용하므로 프로그램 구조를 단순화할 수 있으며 코드 크기도 줄일 수 있다.

- Application -> RTOS API -> Scheduling/timing -> Hardware

○ 유지보수성과 확장성 (Maintainability / Extensibility)

➤ 시간과 관련된 처리를 RTOS가 담당하면 각 소프트웨어 모듈 사이의 의존성을 줄일 수 있다. 따라서 소프트웨어를 보다 체계적이고 예측 가능한 방식으로 수정하고 확장할 수 있다. 또한 하드웨어가 변경되더라도 애플리케이션의 동작 특성이 받는 영향을 줄일 수 있다.

○ 모듈화 (Modularity)

➤ 각 Task는 서로 독립적인 모듈로 설계할 수 있으며, 각각 명확한 목적을 갖도록 구성하는 것이 좋다.

- Task 1: 센서 데이터 수집
- Task 2: 데이터 처리
- Task 3: 통신 처리
- Task 4: 사용자 인터페이스

실시간 운영체제

□ Why Use an RTOS? (계속)

○ 팀 개발에 유리 (Team Development)

- 각 Task가 명확한 인터페이스를 가지도록 설계하면 여러 개발자가 서로 다른 Task를 독립적으로 개발할 수 있다.

○ 테스트가 용이 (Easier Testing)

- 독립적으로 구성된 Task와 명확한 인터페이스는 각 기능을 개별적으로 테스트하기 쉽게 한다.

○ 코드 재사용성 향상 (Code Reuse)

- 모듈화 수준이 높고 다른 모듈과의 의존성이 낮은 코드는 다른 프로젝트에서도 쉽게 재사용할 수 있다.

○ 시스템 효율성 향상 (Improved Efficiency)

- RTOS 기반 프로그램은 Event-Driven 방식으로 설계할 수 있다. 따라서 아직 발생하지 않은 이벤트를 계속 확인하는 Polling 방식에 CPU 시간을 낭비하지 않아도 된다.
- Polling 방식

```
while(1) {  
    센서 확인  
    버튼 확인  
    통신 확인  
    ...  
}
```

실시간 운영체제

□ Why Use an RTOS? (계속)

○ Idle Time 활용

- 실행할 애플리케이션 Task가 없으면 FreeRTOS가 자동으로 생성하는 Idle Task가 실행된다.
 - 남은 CPU 처리 능력 측정
 - 백그라운드 작업 수행
 - 시스템 상태 점검
 - 프로세서를 저전력 모드로 전환

○ 전력 관리 (Power Management)

- RTOS를 사용하면 CPU가 불필요하게 동작하는 시간을 줄일 수 있으므로 프로세서가 저전력 상태에 머무를 수 있는 시간을 증가시킬 수 있다. 특히 FreeRTOS는 Tick-less Mode를 지원하여 CPU가 더 오랫동안 저전력 모드에 머무를 수 있도록 한다.



실시간 운영체제

□ Why Use an RTOS? (계속)

○ 유연한 인터럽트 처리 (Flexible Interrupt Handling)

- 인터럽트 서비스 루틴(ISR)을 가능한 한 짧게 유지하고 실제 처리는 별도의 Task로 넘길 수 있다. FreeRTOS에서는 애플리케이션 Task 또는 자동으로 생성되는 RTOS Daemon Task(Timer Task)로 처리를 연기할 수 있다.



○ 다양한 처리 요구사항 지원 (Mixed Processing Requirements)

- FreeRTOS에서는 하나의 시스템 안에서 다양한 실행 형태를 함께 구현할 수 있다. 또한 Task와 Interrupt의 Priority를 적절히 설정하여 Hard Real-Time과 Soft Real-Time 요구사항을 동시에 처리할 수 있다.
 - Periodic Processing – 주기적 처리
 - Continuous Processing – 연속 처리
 - Event-Driven Processing – 이벤트 기반 처리

실시간 운영체제

□ FreeRTOS Kernel 주요 기능

○ 핵심 기능

Preemptive Scheduling	선점형 스케줄링
Cooperative Scheduling	협력형 스케줄링
Time Slicing	동일 우선순위 Task 시간 분할
Task Priority	우선순위 기반 작업
Task Notification	Task간 경량 통신
Queue	Task간 데이터 전달
Binary Semaphore	동기화
Counting Semaphore	자원 또는 이벤트 개수 관리
Mutex	상호 배제
Recursive Mutex	재귀적 Mutex
Software Timer	소프트웨어 타이머
Event Group	비트 단위 이벤트 관리